

Research Summary: Efficient Secure Three-Party Sorting with Applications to Data Analysis and Heavy Hitters

Paper by: Gilad Asharov, Koki Hamada, Dai Ikarashi, Ryo Kikuchi, Ariel Nof, Benny Pinkas, Katsumi Takahashi, Junichi Tomida (2022)

Hitarth Makawana
Roll No: 230479
hitarthkm23@iitk.ac.in

Ronav Puri
Roll No: 230815
ronavgp23@iitk.ac.in

March 12, 2026

Premise

The paper presents an efficient three-party sorting protocol secure against passive and active adversaries in the honest majority setting. The protocol can be combined with other secure protocols to enable various data analysis tasks.

- **MP Sorting:** The protocol securely sorts m shared items among three servers in the presence of at most one corrupted party without revealing the underlying values.
- **Private Set Intersection:** This protocol can be applied to *Private Set Intersection* problem and its modifications like *Threshold multi-party PSI* and *Data deduplication*.
- **Heavy Hitters & Telemetry:** Applied to privacy-preserving telemetry, clients share data, and servers identify the most frequent strings (e.g., heavy hitters) without leaking individual data.

Ideal Functionalities & Sub-protocols Used

$\mathcal{F}_{rand}$ - Generate shares of a random value

- **Functionality:** $\mathcal{F}_{rand}$ receives corrupt party P_i 's shares (α_i, α_{i+1}) . It generates a random $\alpha \in \mathcal{R}$ and sets $\alpha_{i-1} = \alpha - \alpha_i - \alpha_{i+1}$. It sends the respective shares to the honest parties.
- **Implementation:** It uses *Pseudorandom Secret Sharing (PRSS)*, where each pair of parties holds a shared PRF key. Thus, each P_i computes $\alpha_i := PRF_{key_i}(ctr)$ and $\alpha_{i+1} := PRF_{key_{i+1}}(ctr)$ for some online counter ctr .

$\mathcal{F}_{mult}$ - Multiplication

- **Functionality:** $\mathcal{F}_{mult}$ receives corrupt party P_i 's shares (c_i, c_{i+1}) as well each party's shares of a and b . It reconstructs a and b , computes $c = ab$ as well as $c_{i-1} = c - c_i - c_{i+1}$. It sends the respective shares to the honest parties.
- **Implementation:** It can be realized with passive security via the multiplication protocol of *High-Throughput Semi-Honest Secure Three-Party Computation with an Honest Majority* by Toshinori Araki, Jun Furukawa, Yehuda Lindell, Ariel Nof, Kazuma Ohara (2016)

Secure Shuffling Protocol

Algorithm 1 RESHARE

Input: A secret-shared value $\llbracket \mathbf{x} \rrbracket$ with shares x_1, x_2, x_3 , a target party index $\mathbf{i}$.

Output: Party P_i receives new randomized shares of x .

Parties P_{i-1} & P_{i+1} call $\mathcal{F}_{rand}$ to choose random values r_1, r_2 , and r_3 such that $r_1 + r_2 + r_3 = 0$.

P_{i-1} & P_{i+1} locally compute $x'_1 := x_1 + r_1$, $x'_2 := x_2 + r_2$, and $x'_3 := x_3 + r_3$.

P_{i-1} sends x'_i to P_i .

P_{i+1} sends x'_{i+1} to P_i .

-
- In the REVEAL protocol, all parties collude together and share their secret shares of a value $\llbracket x \rrbracket$ with one another. Hence, all parties can compute the complete value x . In other words, x is publicly revealed or announced.
-

Algorithm 2 SHUFFLE

Input: A secret-shared permutation $\langle \langle \pi \rangle \rangle$ such that $\pi = \pi_3 \circ \pi_2 \circ \pi_1$, a shared vector $\llbracket \vec{\mathbf{a}} \rrbracket$ such that $\vec{\mathbf{a}} = (a_1, \dots, a_m)$.

Output: The secret-shared shuffled vector $\llbracket \vec{\mathbf{b}} \rrbracket = \llbracket \pi(\vec{\mathbf{a}}) \rrbracket$

Let $\vec{\mathbf{b}}_0 = \vec{\mathbf{a}}$.

for $i = 1$ **to** 3 **do**

P_{i-1} and P_i define $\llbracket \vec{\mathbf{b}}_i \rrbracket = \llbracket [\pi_i(\vec{\mathbf{b}}_{i-1})] \rrbracket$.

The parties run RESHARE($\llbracket \vec{\mathbf{b}}_i \rrbracket, i + 1$) .

end for

The parties set $\vec{\mathbf{b}} := \vec{\mathbf{b}}_3$.

return $\llbracket \vec{\mathbf{b}} \rrbracket$.

-
- **Communication Complexity:** Overall communication per party = $2m$ ring elements.
 - The UNSHUFFLE protocol, as the name suggests, un-shuffles the shuffled vector $\llbracket \vec{\mathbf{b}} \rrbracket = \llbracket \pi(\vec{\mathbf{a}}) \rrbracket$. It's pseudocode is analogous to the SHUFFLE protocol.
-

Sorting Algorithm

The authors utilize a stable sort based on the **radix sort**, which is asymptotically better than previous secure sorting protocols. Instead of shuffling the entire length of the items after each comparison step, it computes a composition of permutations.

Characteristics & Primitive

- **Data-Oblivious:** The sequence of comparisons is independent of input values, preventing side-channel leakage. Instead of using fixed routing paths, obliviousness is achieved by securely computing secret-shared permutations bit-by-bit.
- **Stable Sorting:** Unlike other sorting algorithms used in secure MP sorting protocols, radix sort provides a stable sorting of elements.
- **Structure:** The algorithm is built on bitwise radix sort processing starting from the rightmost LSB. It uses a secure COMPOSE protocol to mathematically merge the routing instructions of every single bit into one single *master permutation*.

Algorithm 3 GENBITPERM - Generating permutation of stable sort for a single bit key

Input: Secret-shared bit-wise keys $\llbracket \vec{k} \rrbracket$, where $k_i \in \{0, 1\} \forall i \in \{1, \dots, m\}$.

Output: The secret-shared permutation $\llbracket \rho \rrbracket$ which is a stable sorting of $\vec{k}$

Complexity per party: $m||\mathcal{R}||$ bits

```
for  $1 \leq i \leq m$  do
   $\llbracket f_i^{(0)} \rrbracket := 1 - \llbracket k_i \rrbracket$ .
   $\llbracket f_i^{(1)} \rrbracket := \llbracket k_i \rrbracket$ .
end for
 $\llbracket s \rrbracket := \llbracket 0 \rrbracket$ .
for  $j = 0$  to  $1$  do
  for  $i = 1$  to  $m$  do
     $\llbracket s \rrbracket := \llbracket s \rrbracket + \llbracket f_i^{(j)} \rrbracket$ .
     $\llbracket s_i^{(j)} \rrbracket := \llbracket s \rrbracket$ 
  end for
end for
for  $1 \leq i \leq m$  do
  Send  $(\llbracket k_i \rrbracket, \llbracket s_i^{(1)} \rrbracket - \llbracket s_i^{(0)} \rrbracket)$  to  $\mathcal{F}_{mult}$  and receive  $\llbracket t_i \rrbracket$ .
   $\llbracket \rho(i) \rrbracket := \llbracket s_i^{(0)} \rrbracket + \llbracket t_i \rrbracket$ 
end for
return  $\llbracket \rho \rrbracket = (\llbracket \rho(1) \rrbracket, \dots, \llbracket \rho(m) \rrbracket)$ 
```

Algorithm 4 APPLYPERM - Applying a shared-vector permutation

Input: A secret-shared permutation $\llbracket \rho \rrbracket$, a secret-shared vector $\llbracket \vec{k} \rrbracket$.

Output: A secret-shared vector $\llbracket \vec{k}' \rrbracket$ such that $\vec{k}' = \rho(\vec{k})$.

Complexity per party: $5m||\mathcal{R}||$ bits

```
Call  $\mathcal{F}_{rand}$  and receive  $\langle \langle \pi \rangle \rangle$ .
 $\llbracket \pi(\vec{\rho}) \rrbracket \leftarrow \text{SHUFFLE}(\langle \langle \pi \rangle \rangle, \llbracket \vec{\rho} \rrbracket)$ 
 $\llbracket \pi(\vec{k}) \rrbracket \leftarrow \text{SHUFFLE}(\langle \langle \pi \rangle \rangle, \llbracket \vec{k} \rrbracket)$ 
Call REVEAL( $\llbracket \pi(\vec{\rho}) \rrbracket$ ) to obtain  $\pi(\vec{\rho}) = \rho \circ \pi^{-1}$ .
Locally apply  $\rho \circ \pi^{-1}$  on  $\llbracket \pi(\vec{k}) \rrbracket$  to obtain  $\llbracket \vec{k}' \rrbracket = \llbracket \rho(\vec{k}) \rrbracket$ .
return  $\llbracket \vec{k}' \rrbracket$ 
```

Algorithm 5 COMPOSE - Composition of two share-vector permutations

Input: Secret-shared permutations $\llbracket \vec{\sigma} \rrbracket, \llbracket \vec{\rho} \rrbracket$.

Output: Secret-shared permutation $\llbracket \vec{\tau} \rrbracket$, where $\tau = \rho \circ \sigma$.

Complexity per party: $5m||\mathcal{R}||$ bits

```
Call  $\mathcal{F}_{rand}$  and obtain  $\langle \langle \pi \rangle \rangle$ .
 $\llbracket \pi(\vec{\sigma}) \rrbracket \leftarrow \text{SHUFFLE}(\langle \langle \pi \rangle \rangle, \llbracket \vec{\sigma} \rrbracket)$ 
Call REVEAL( $\llbracket \pi(\vec{\sigma}) \rrbracket$ ) to obtain  $\pi(\vec{\sigma}) = \sigma \circ \pi^{-1}$ .
Locally apply  $(\sigma \circ \pi^{-1})^{-1} = \pi \circ \sigma^{-1}$  to  $\llbracket \vec{\rho} \rrbracket$  to obtain  $\llbracket \vec{\gamma} \rrbracket = \llbracket \pi \circ \sigma^{-1}(\vec{\rho}) \rrbracket$ .
 $\llbracket \vec{\tau} \rrbracket \leftarrow \text{UNSHUFFLE}(\langle \langle \pi \rangle \rangle, \llbracket \vec{\gamma} \rrbracket)$ 
return  $\llbracket \vec{\tau} \rrbracket$ 
```

Algorithm 6 GENPERM - Securely Generating a Stable Sorting Permutation

Input: Secret-shared keys $\llbracket \vec{k} \rrbracket = (\llbracket k_1 \rrbracket, \dots, \llbracket k_m \rrbracket)$ where $\llbracket k_i \rrbracket$ has bits $(\llbracket k_i^{(1)} \rrbracket, \dots, \llbracket k_i^{(l_k)} \rrbracket)$.
Output: Permutation $\llbracket \vec{\sigma} \rrbracket$ for stable sorting $\vec{k}$
Complexity per party: $11m \cdot |\mathcal{R}| \cdot l_k$ bits

For each $j \in [l_k]$, let $\llbracket \vec{k}^{(j)} \rrbracket = (\llbracket k_1^{(j)} \rrbracket, \dots, \llbracket k_m^{(j)} \rrbracket)$.

$\llbracket \vec{\rho}_1 \rrbracket \leftarrow \text{GENBITPERM}(\llbracket \vec{k}^{(1)} \rrbracket)$

$\llbracket \vec{\sigma}_1 \rrbracket := \llbracket \vec{\rho}_1 \rrbracket$

for $j = 2$ **to** l_k **do**

$\llbracket \vec{k}'^{(j)} \rrbracket \leftarrow \text{APPLYPERM}(\llbracket \vec{\sigma}_{j-1} \rrbracket, \llbracket \vec{k}^{(j)} \rrbracket)$

$\llbracket \vec{\rho}_j \rrbracket \leftarrow \text{GENBITPERM}(\llbracket \vec{k}'^{(j)} \rrbracket)$

$\llbracket \vec{\sigma}_j \rrbracket \leftarrow \text{COMPOSE}(\llbracket \vec{\sigma}_{j-1} \rrbracket, \llbracket \vec{\rho}_j \rrbracket)$

end for

return $\llbracket \vec{\sigma}_{l_k} \rrbracket$

- **Overall communication complexity per party:** $11m \cdot |\mathcal{R}| \cdot l_k + 3m \cdot |\mathcal{R}| + 2m \cdot |\mathcal{R}'|$ bits
(Upper bound = $14l_k \cdot m \log m + 2l_p m$ bits)

$\mathcal{F}_{\text{sort}}$ - Stable sorting functionality

- **GenPerm Functionality:** $\mathcal{F}_{\text{sort}}$ receives all shares of $\llbracket \vec{k} \rrbracket$ from the honest parties. It reconstructs $\vec{k}$ and computes the permutation σ which corresponds to the stable sorting of $\vec{k}$. It then secret shares $\llbracket \sigma \rrbracket$ amongst the parties.
- **ApplyPerm Functionality:** $\mathcal{F}_{\text{sort}}$ receives all shares of $\llbracket \sigma \rrbracket$ and $\llbracket \vec{v} \rrbracket$ from the honest parties. It reconstructs σ and $\vec{v}$ and computes $\vec{v}' = \sigma(\vec{v})$. It then secret shares $\llbracket \vec{v}' \rrbracket$ amongst the parties.

Modifications for Malicious Security

- **Authenticated secret sharing:** The protocol is modified by adding a MAC to each secret, meaning parties hold $(\llbracket x \rrbracket, \llbracket r \cdot x \rrbracket)$ for a random, unknown r . All operations done on $\llbracket x \rrbracket$ will also be done on $\llbracket r \cdot x \rrbracket$. The MAC will be used for verification to detect cheating at the end of any operation.
- **Security upto additive attacks:** Semi-honest protocols are modified to be secure up to additive attacks, where a malicious adversary can only introduce an independent offset error to the output.
- $\mathcal{F}_{l\text{-mult}}^{\text{add}}$ **functionality:** An extended inner-product functionality that incorporates an additive error Δ chosen by the adversary.
- $\mathcal{F}_{\text{reshare-shuffle}}^{\text{add}}$ **functionality:** Models resharing after a local shuffle, allowing a corrupted party to inject an additive error into their reshared output. Even though it cannot guarantee the correctness of output in presence of malicious adversary, the verification of MAC tags at the end of the protocol will ensure that cheating is detected.
- **Modifications to the semi-honest protocol:**
 1. *Input randomization:* The original input $\llbracket \vec{k} \rrbracket$ is pre-processed to attach a MAC with every input element. Thus, the new input becomes $(\llbracket \vec{k} \rrbracket, \llbracket r \cdot \vec{k} \rrbracket)$.
 2. *Modified GENPERM:* The parties run GENPERM on both $\llbracket \vec{k} \rrbracket$ and $\llbracket r \cdot \vec{k} \rrbracket$. Each $\mathcal{F}_{\text{mult}}$ call is replaced with $\mathcal{F}_{l\text{-mult}}^{\text{add}}$ with $l = 1$. Every SHUFFLE call is accompanied by an $\mathcal{F}_{\text{reshare-shuffle}}^{\text{add}}$ call. Whenever there is a REVEAL call, the parties first run the verification protocol.

Implementation & Results

- **Implementation Settings:** The protocols were implemented in **C++11**, taking advantage of multi-threading and AES-NI on **AWS c5.18xlarge** servers.
- **Scheme: Linear (additive) secret sharing** over the ring $\mathbb{Z}_p$ ($p = 2^{61} - 1$).
- **Results:** The implementation proves concretely efficient, sorting one million 32-bit records in 2.26 seconds with semi-honest security and 5.89 with malicious security ($\lambda = 32$).

Improving Concrete Efficiency

- **Batch processing:** Concrete efficiency is improved by processing multiple bits at a time (e.g., $L = 3$) in GENMULTIBITSORT, reducing round complexity.
- **Reusing permutations:** The same random permutations can be reused across APPLYPERM and COMPOSE steps within a single iteration to minimize communication overhead.
- **Permuting small shares:** Communication is reduced by operating on small shares in $\mathbb{Z}_2$ rather than a large ring during APPLYPERM, utilizing modulus conversion before proceeding.
- **Optimized shuffling:** P_1 knows π_2 and π_1 that used for the first and second shuffling steps. The randomness used in resharing also can be obtained via $\mathcal{F}_{rand}$ beforehand. Therefore, P_1 can permute $\vec{a}$ using π_2 and π_1 at once.